Shor’s Error Detecting Quantum Algorithm Hardware Accelerator

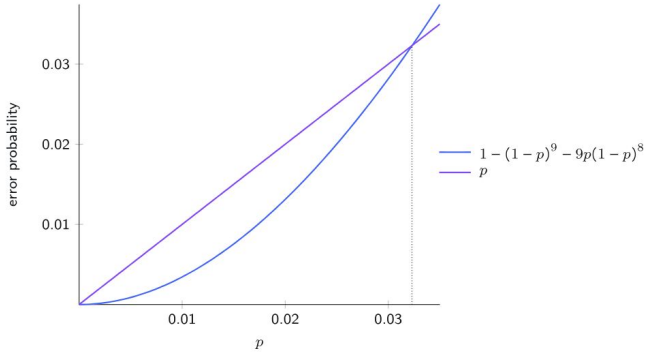
By: Shayna Messler, Owen Delaney, Leo Silverman

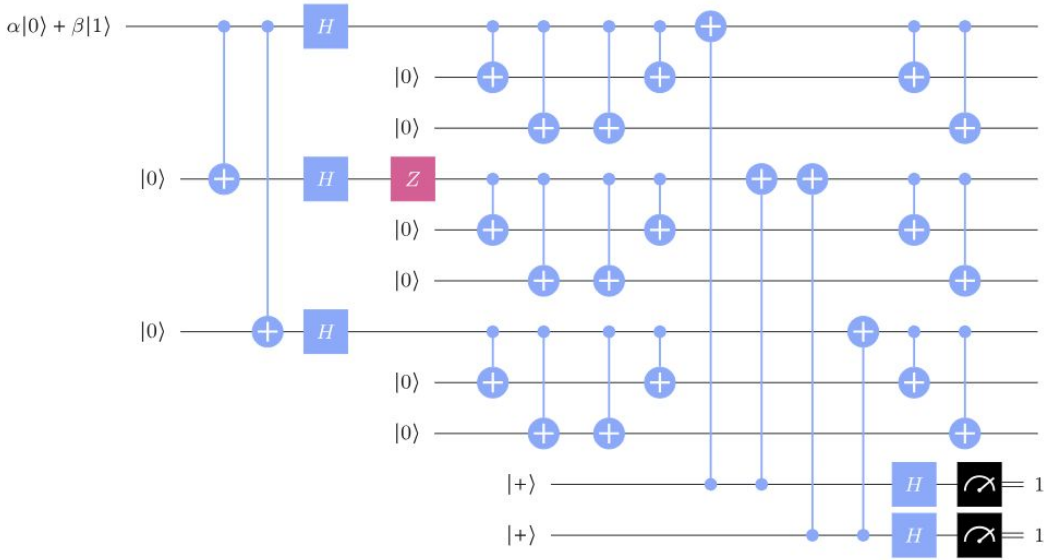
Background

- With the growth in interest in quantum computing along with the rise of AI, simulating quantum computations has come to the forefront, because of the current unreliability of physical quantum computers.
- The impetus for this project is the fact that qubit states grow at a rate of 2^n, rather than in classical bits, that grow at 2*n.
- There is a limited number of qubits in the world, and doing a large amount of quantum testing is difficult.

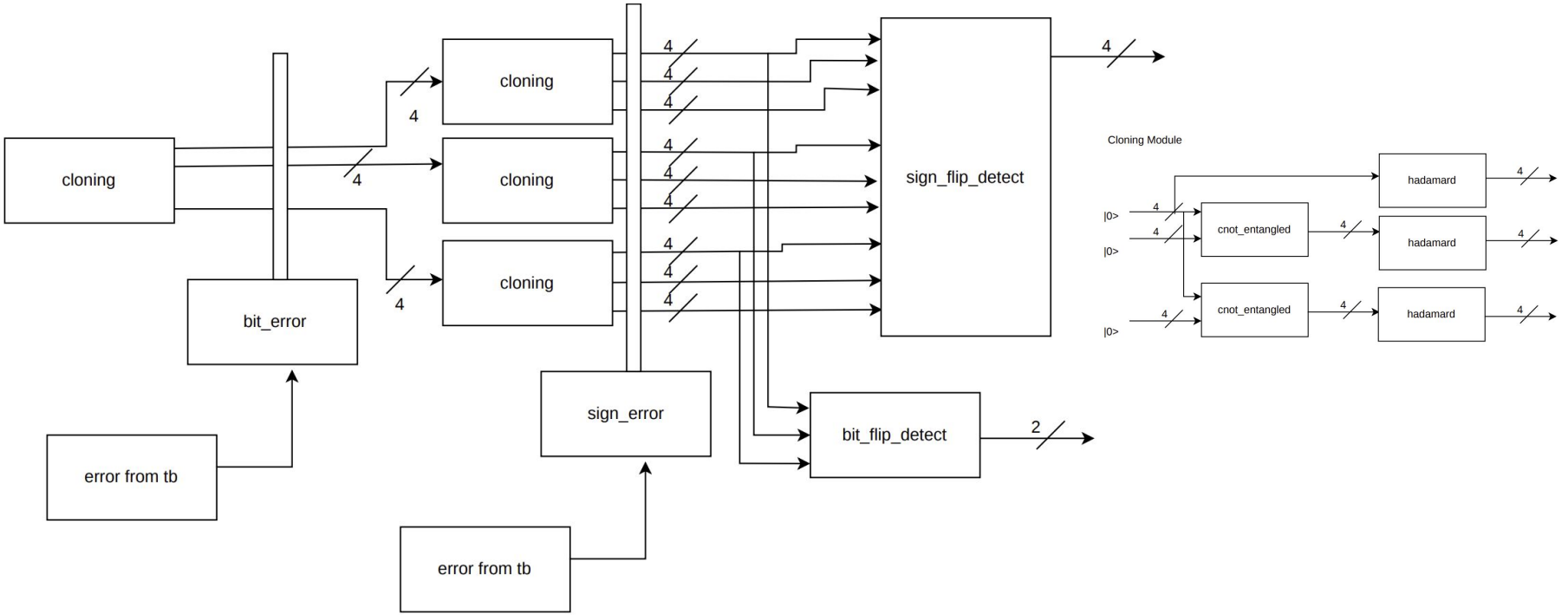
How Shor's Error Detecting Algorithm Work? [1]

- Quantum Computing No-Cloning Theorem
- Measurement and Additional Error Detection Bits
- Noise Level Probability of Program Working

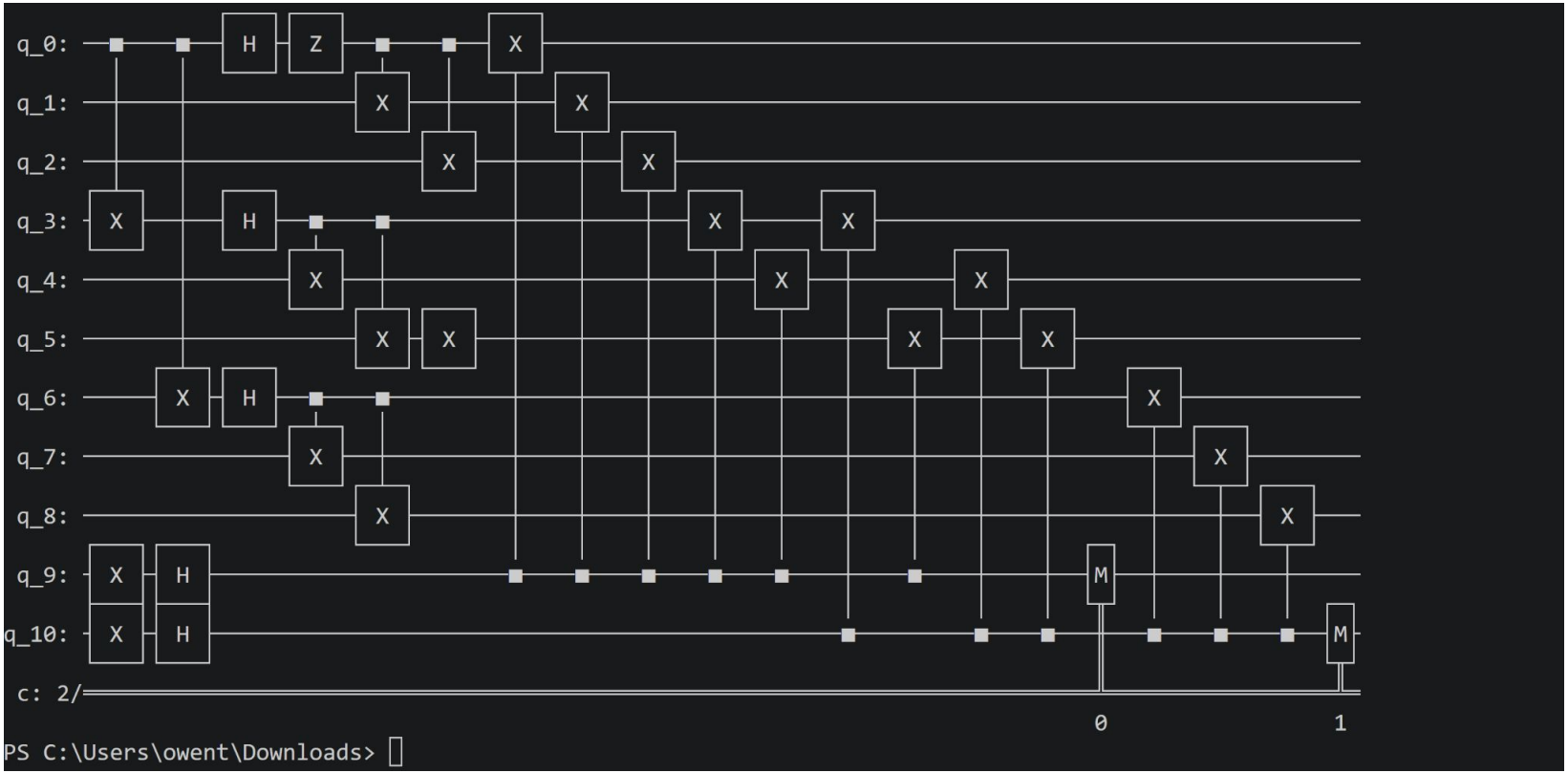




RTL Diagram


cloning
bit_error
error from tb
cloning
cloning
cloning
sign_error
error from tb
sign_flip_detect
bit_flip_detect
4
2
Cloning Module
|0>
|0>
|0>
cnot_entangled
cnot_entangled
hadamard
hadamard
hadamard

Qiskit (python) Simulation example



Cloning (CNOT and Hadamard)

```
module cnot_entangled (
    input  logic signed [1:0] q1 [2][1],
    input  logic signed [1:0] q2 [2][1],
    output logic signed [1:0] q1ent [2][1],
    output logic signed [1:0] q2ent [2][1]
);

    assign q1ent = q1;

    always_comb begin
        if (q1[1][0] != 2'sd0) begin
            q2ent[0][0] = q2[1][0];
            q2ent[1][0] = q2[0][0];
        end else begin
            q2ent = q2;
        end
    end

endmodule
```

```
module cloning (
  input  logic signed [1:0] in1 [2][1], in2 [2][1], in3 [2][1],
  output logic signed [1:0] out1 [2][1], out2 [2][1], out3 [2][1]
);

  localparam logic signed [1:0] HGATE [2][2] = '{
      '{ 2'sd1,  2'sd1},
      '{ 2'sd1, -2'sd1}
  };

  logic signed [1:0] int1 [2][1], int2 [2][1], int3 [2][1];
  logic signed [1:0] d1 [2][1], d2 [2][1];

  cnot_entangled first  (.q1(in1), .q2(in2), .q1ent(int1), .q2ent(d1));
  cnot_entangled second (.q1(in2), .q2(in3), .q1ent(int2), .q2ent(d2));

  assign int3 = in3;

  matrixMultiply2x2 H1 (.A(HGATE), .B(in1),  .result(out1));
  matrixMultiply2x2 H2 (.A(HGATE), .B(int2), .result(out2));
  matrixMultiply2x2 H3 (.A(HGATE), .B(int3), .result(out3));

endmodule
```

Error Injection

- Synthetic errors were simulated, both in bit flips and phase flips.
- The module would include the bit the error would occur on in an input wire.

```
function automatic void apply_x_flip (
    input  logic signed [1:0] q_in  [2][1],
    output logic signed [1:0] q_out [2][1]
);
    q_out[0][0] = q_in[1][0];
    q_out[1][0] = q_in[0][0];
endfunction
```

```
function automatic void apply_z_flip (
    input  logic signed [1:0] q_in  [2][1],
    output logic signed [1:0] q_out [2][1]
);
    q_out[0][0] =  q_in[0][0];
    q_out[1][0] = -q_in[1][0];
endfunction
```

Bit and Phase Flip Detecting

- Detection qubits allows for the working qubits to be unaffected by the measurement (state collapse) to detect bit and phase flips.
- A detection of a bit flip, will switch the sign of the superposition state and with a hadamard, will collapse into the one state, showing an error.

```
case (counter)
    // Step 0: First parity CNOT (q1 -> e1, q2 -> e2)
    3'd0: begin
        e1 <= c11;
        e2 <= c21;
    end
    // Step 1: Second parity CNOT (q2 -> e1, q3 -> e2)
    3'd1: begin
        e1 <= c12;
        e2 <= c22;
    end
    // Step 2: Latch syndrome from |1> amplitude of e1 and e2
    // e1[1][0] is parity(q1, q2), e2[1][0] is parity(q2, q3)
    3'd2: begin
        error <= {e1[1][0][0], e2[1][0][0]};
    end
    default: begin
        // Hold output steady until next rst_n pulse
        counter <= 3'd3;
    end
endcase
```

Test Bench Results

Tested:

Ideal State (No Error): Injecting Error Code = 2'b00

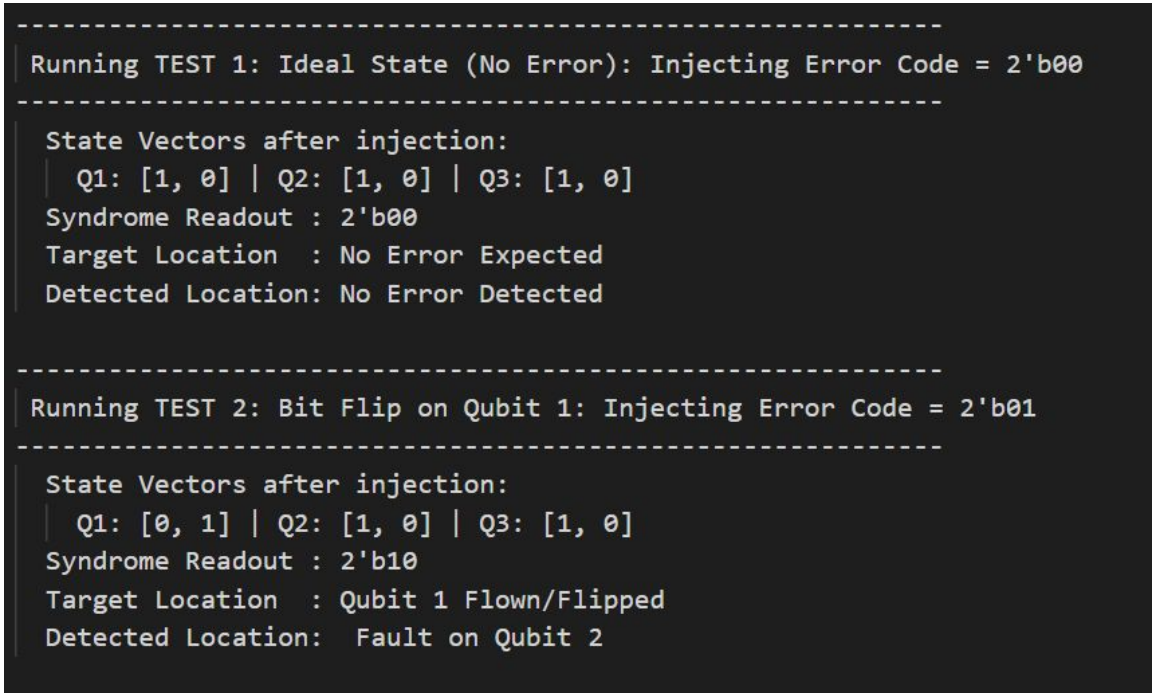
Bit Flip on Qubit 1: Injecting Error Code = 2'b01

Bit Flip on Qubit 2: Injecting Error Code = 2'b10

Bit Flip on Qubit 3: Injecting Error Code = 2'b11

Bit Flip on Qubit 1 in |111>: Injecting Error Code = 2'b01

Bit Flip on Qubit 2 in |111>: Injecting Error Code = 2'b10



Why it is Important?

- Growing quantum computing, and quantum computers, especially if AI will grow through quantum computing, requires large amount of simulations.
- This project was a test to see how feasible offloading quantum simulations is to FPGAs and how we could implement that.
- Our program uses 2x1 vectors of signed bits to represent the qubits.
- We keep down the memory by only manipulating those vectors and representing entanglement with a smaller amount of bits.
- Instead of the 512 bits necessary to perform this operation, we used 36 bits. (Not including the measurement bits)

Works Cited:

[1]
“The 9-qubit Shor code | IBM Quantum Learning,” *IBM Quantum Learning*, 2017. https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/correcting-quantum-errors/shor-code (accessed Sept. 20, 2026).